TEAM ENABLEMENT SESSION · 45 MINUTES

AI-Augmented SDLC Framework

How we use AI at every stage — and how we prove, by manual review and recorded gates, that every output is correct, consistent and traceable.

Web + Microservices

Mobile Application

Internal Web Apps

Fixed Assets · Depreciation · Labelling and Tracking · Data Validation and Audit

AGENDA

What we will cover in the next 45 minutes

Where we stand	The process we had, and what AI outran	4 min
Pros, cons and the control for each	Honest view, plus the control that removes each risk	6 min
The framework	Nine stages, one repeatable pattern per stage	5 min
The common context layer	How every developer gets the same output from AI	7 min
Stage by stage	Inputs, AI activity, manual review, exit gate, roles	11 min
Verification and governance	Traceability, gates, RACI, metrics	6 min
Rollout and decisions	30-60-90 plan and what we need from you	4 min

Questions welcome throughout — decisions we need from you are listed on the final slide.

01

Where we stand

The process that kept us consistent, and the checkpoints AI outran

WHERE WE STAND

The discipline we had, and the steps AI quietly outran

THE MANUAL PROCESS THAT KEPT US CONSISTENT

- Functional Requirement Document signed before build
- Coding and naming standards, verified by PMD and Sonar
- One project per feature, fixed folder structure
- Common base classes and shared components for reuse
- Test scenarios and test cases written, then automated
- Issues tracked in Jira against a requirement
- Deployment only through Jenkins pipelines

WHAT HAPPENS NOW THAT AI DRAFTS FIRST

- Requirement goes straight into a prompt — no signed document
- AI picks its own patterns; our standards are never loaded
- Each developer scaffolds a different structure
- New utilities and CSS written while base classes sit unused
- Test cases generated, but not reconciled with what was automated
- Commits without a requirement ID — traceability matrix drifts
- Pipeline still runs, but it no longer checks what it used to

AI did not break our process. It outran the checkpoints that used to catch us — so we put the checkpoints back, at AI speed.

WHERE WE STAND

Six gaps that opened — and what each one costs us

Code inconsistency

The same problem solved three different ways across service, mobile and internal web.

Standards deviation

Generic AI patterns replace our naming, layering, logging and error handling.

Lost reuse

New utilities, validators and CSS written while proven base classes sit unused.

Traceability gaps

Code, tests and defects with no requirement ID. Coverage cannot be proven at audit.

Silent omissions

Requirements and test cases dropped during generation, with nothing to flag the loss.

Review debt

Large AI diffs approved quickly; the defects surface after release, not before.

The compounding cost: every inconsistency we accept today is paid again at every future enhancement. An inconsistent codebase turns routine change requests into critical, high-risk projects — which is exactly what this framework exists to prevent.

02

Pros, cons and the control for each

An honest view of AI, and the named control that removes every risk

What AI genuinely gives us — this is why we keep it

Speed on first drafts

SFVD, boilerplate, test cases, release notes and scripts arrive in minutes, not days.

Breadth of coverage

Boundary, negative and cross-surface scenarios that a tired human reliably forgets.

Consistency when context is pinned

Given the same standards and examples, AI repeats our patterns more faithfully than we do.

Documentation stops lagging

Traceability entries, API docs and release notes generated from the source of truth.

Levelling across the team

A first-year developer produces output close to the team standard, if the context is right.

Senior time moves upstream

Architects spend their hours on design, reuse and review instead of boilerplate.

None of this is in dispute. The question is not whether to use AI — it is what we put around it.

Where AI fails us — and why it fails quietly

1. **Confident but wrong**
   Invents fields, validations, error codes and API shapes that look entirely plausible.

2. **Non-deterministic**
   Two developers, one requirement, two different designs — both defensible, neither aligned.

3. **Context-blind by default**
   It cannot see our base classes, so it rewrites them instead of calling them.

4. **Silent omission**
   Drops a requirement or a test case and never tells you what it left out.

5. **Over-generation**
   Dead code, speculative abstractions and brand-new CSS nobody asked for.

6. **Traceability loss**
   Output arrives with no link to a requirement, so coverage cannot be proven.

7. **Security and IP exposure**
   Secrets pasted into prompts; snippets of unknown licence pulled into our product.

8. **Review fatigue**
   A 900-line generated diff gets the same five minutes a 90-line one used to get.

Every one of these is manageable. None of them is self-correcting — each needs a named control and a named human.

Every risk has one named control, one stage and one piece of evidence

Risk from AI	The control that removes it	Stage	Evidence we keep
Confident but wrong output	Checklist review against the data dictionary, validation and error-code catalogue. AI must declare every assumption it made.	S1-S3	Signed review checklist
Different design each time	Pinned Context Pack, standard stage prompt templates and golden reference examples.	All	Context version stamped on the artifact
Rewrites what we already have	Shared library index loaded as context; reuse question is mandatory in review; duplication gate in CI.	S3-S5	Sonar duplication report
Coding and naming drift	Standards files in context; PMD, Checkstyle and naming lint block the pull request.	S4-S5	CI gate result on the PR
Silent omission	Traceability link check and automation parity check — approved cases must equal executed cases.	S1, S6, S7	RTM completeness report
Traceability loss	Requirement ID mandatory in branch, commit, test case, defect and release note.	All	Automated ID link-check job
Over-generation and dead code	Reuse-before-create rule, style class catalogue, reviewer removes anything unused.	S4-S5	Review comments on the PR
Secrets and IP exposure	Approved tooling only, no secrets in prompts, dependency and secret scanning in CI.	All	Scan report and tooling policy
Review fatigue	Pull requests under 400 lines, independent reviewer, monthly audit of a 10% sample.	S5	Audit log and PR size metric

Not one of these controls requires us to trust the AI. Each one is a check a person or a job performs on the output.

OPERATING MODEL

Five principles that govern every stage

AI drafts. A human decides. The gate records.

AI output is never a deliverable. It becomes one only when a named reviewer accepts it and the gate is recorded.

Context is shared, never personal

Standards, structure, shared libraries, style classes and test data live in one versioned pack that every prompt loads.

Nothing moves without an ID

Every artifact, commit, test case and defect carries the requirement ID. If it is not linked, it does not ship.

Reuse before create

A new utility, base method, CSS class or service is justified in writing, or it is not written at all.

Evidence or it did not happen

A gate passes only with a recorded artifact: checklist, scan result, approval. Verbal sign-off is not a gate.

03

The framework

Nine stages, one repeatable pattern, no output without a gate

THE FRAMEWORK

Nine stages: what AI does, and the gate that closes each one

1 Requirements

AI Structures the register, lists every ambiguity

GATE Register approved, requirement IDs frozen

2 SFVD

AI Drafts screens, fields, validations, flows

GATE SFVD baselined, field matrix updated

3 Design and contract

AI Drafts API contract, impact and reuse map

GATE Contract merged before any code starts

4 Development

AI Writes code inside our structure and libraries

GATE PR with checklist, static gates green

5 Code review

AI Scans for deviation, duplication, missing tests

GATE Independent human approval recorded

6 Test design

AI Generates cases from SFVD and contract

GATE At least one case per requirement ID

7 Test execution

AI Drafts automation scripts and defect notes

GATE Automation parity check passes

8 Build and release

AI Drafts release notes from commits and IDs

GATE Signed, immutable release package

9 Deploy and close

AI Builds smoke pack, summarises post-deploy logs

GATE Smoke passes, traceability matrix frozen

Mandatory manual review sits between the AI column and the gate column in every single stage. No stage output becomes the next stage input without a named reviewer and a recorded gate.

THE FRAMEWORK

One pattern, repeated in all nine stages — learn it once

Stage N

becomes the input to Stage N+1

Approved input

The signed artifact from the previous stage, plus the pinned Context Pack version.

Standard prompt

The stage prompt template. Nobody writes a free-form prompt for a deliverable.

AI draft

Deliverable in the standard template, with assumptions and open questions declared.

Manual review

Four levels: automated checks, role checklist, independent reviewer, recorded sign-off.

Signed artifact

Versioned, context-stamped, linked in the traceability matrix, released to the next stage.

Rejected at the gate: the artifact returns to the same stage with a recorded reason — and the gap is raised as a pull request against the Context Pack, so the same mistake is not regenerated by anyone else tomorrow.

- **Nothing skips a box**
- **The reviewer is never the AI operator**
- **The gate is a record, not a conversation**

04

The common context layer

How every developer gets consistent output from AI, without being told twice

COMMON CONTEXT LAYER

Consistent output is an input problem, not an AI problem



The three layers below the task prompt are shared and versioned — identical for everyone.

WHY THIS FIXES INCONSISTENCY

- Most of what AI needs is identical for every developer and every task — only the last layer changes.
- Pin the shared layers once and two developers asking the same question get the same answer.
- Leave them unpinned and every prompt silently invents its own standard.
- The pack is loaded automatically from the repository, so nobody has to remember to attach it.
- A context gap is treated as a defect: raise a pull request against the pack and everyone is fixed at once.

Same context + same prompt template + same approved input

equals the same shape of output, from every developer, every time.

COMMON CONTEXT LAYER

The Context Pack: one versioned folder, loaded into every prompt

HOW WE BUILD

10-coding-standards.md layering, exceptions, logging, transactions, audit

11-naming-conventions.md package, class, method, table, column, API path, branch

12-folder-structure.md the canonical project skeleton and where a feature goes

22-api-contract-standards.md REST conventions, versioning, paging, error envelope

WHAT WE REUSE

20-shared-library-index.md every base class, validator and utility, with "do not rewrite" list

21-ui-style-guide.md design tokens, approved style classes, component catalogue

99-golden-examples/ one reference feature per surface for AI to imitate

HOW WE PROVE IT

30-test-standards.md case template, ID convention, automation and locator patterns

31-test-data-and-seed.md seed data structure, standard asset fixtures, masking rules

60-review-checklists/ the per-stage checklist that each gate is scored against

DOMAIN AND GUARDRAILS

00-product-overview.md asset classes, depreciation methods, label and audit glossary

40-security-and-compliance.md authorisation model, PII handling, audit log, secrets policy

50-prompt-templates/ the approved prompt for each of the nine stages

Lives at /ai-context in the repository. Loaded automatically by the project instruction file · every change is a pull request with two approvals and a version bump · the version is stamped on every artifact AI produces.

COMMON CONTEXT LAYER

One pack, three surface profiles — the shared sections stay identical

WEB AND MICROSERVICES

- Controller / service / repository layering, enforced by lint
- Base classes: BaseEntity, BaseService, AuditAware, error envelope
- OpenAPI contract first — no handwritten client code
- Package naming per bounded context; no cross-service database access
- Migration script naming and rollback expectations
- Transaction boundary and audit log rules for asset and depreciation writes

MOBILE APPLICATION

- Offline-first and sync rules for asset scanning in the field
- Shared barcode and RFID scan component — never re-implemented
- Screen, navigation and resource naming conventions
- Supported device and OS matrix for generated test cases
- Services consumed only through the generated contract client
- Secure storage, permission and image-capture standards

INTERNAL WEB APPLICATIONS

- Shared component library: grid, filter, upload, date, approval widget
- Approved style classes only — new CSS requires written justification
- Role-based menu and authorisation pattern
- Standard report, export and bulk-upload validation templates
- Audit trail display standard for validation and audit screens
- Pagination and large-dataset handling rules for asset registers

The three surfaces differ only where they are allowed to differ. Domain glossary, requirement IDs, validation rules, error codes, test data and review checklists are the same file for all three — which is what keeps a loosely coupled product behaving like one product.

COMMON CONTEXT LAYER

The standard stage prompt — eight blocks, no free-form prompting

1 ROLE

You are a senior engineer on the Fixed Assets suite, working on the <surface> profile.

2 CONTEXT

Load /ai-context version 2.3, profile: microservice. Treat it as binding, not advisory.

3 INPUT

Approved artifact ID and version — referenced, never retyped or summarised by hand.

4 TASK

One stage deliverable only. Never combine two stages in a single prompt.

5 CONSTRAINTS

Reuse these listed classes and style classes. Do not create new utilities, CSS or error codes.

6 OUTPUT FORMAT

Fill the standard template exactly, so that review becomes mechanical, not interpretive.

7 TRACEABILITY

Stamp the requirement IDs and the context version on every section you produce.

8 SELF-CHECK

List the standards you applied, the reuse decisions you made, and every assumption.

Block 8 is non-negotiable. The declared assumptions list is where the reviewer looks first. It turns "confidently wrong" from an invisible risk into a written claim that a human can accept or reject in seconds.

The nine stage prompts live in /ai-context/50-prompt-templates/ — copy, fill the angle brackets, run. Changing a template is a pull request, not a personal preference.

05

Stage by stage

Inputs to AI, what AI produces, the manual review, the exit gate and the roles

Requirements intake and review points

INPUTS GIVEN TO AI

- Business requirement note and the client review points
- Current workflow export, screen inventory and report list
- Asset glossary: asset classes, SLM and WDV, CWIP, label format
- Integration points and validation rules from the legacy module
- Non-functional targets: volume, response time, audit retention
- Context Pack: product overview and compliance sections

WHAT AI PRODUCES

- Requirement register with a unique ID for every line
- Draft acceptance criteria for each requirement
- Ambiguity and assumption list, declared and never guessed
- Impact map across microservices, mobile and internal web
- First-cut priority and release grouping for discussion

MANUAL REVIEW (MANDATORY)
Business Analyst with Product Owner

- Every source line traced to a requirement ID, nothing dropped
- Ambiguities answered by the business, never by the AI
- Depreciation and statutory rules verified against policy
- Scope boundary and priority agreed with the client
- No requirement invented or silently widened by AI
- Stage checklist completed and attached to the epic

EXIT GATE Requirement Register v1.0 approved in Jira · requirement IDs frozen · traceability matrix rows created · Context Pack version stamped on the document.

ROLES R Business Analyst A Product Owner C Architect, QA Lead I Development, Release

SFVD generation: screens, fields and validations

INPUTS GIVEN TO AI

- Approved requirement register with frozen IDs
- SFVD template plus two approved SFVDs as worked examples
- Naming conventions and UI style guide from the Context Pack
- Validation library and error code registry
- Data dictionary and entity relationship model
- Screen and field inventory from the existing product

WHAT AI PRODUCES

- SFVD draft per screen: purpose, actors and business flow
- Field table: type, length, mandatory, validation ID, error code
- User stories with acceptance criteria per requirement ID
- Reuse callouts naming the components and validators that apply
- Open questions list routed back to the business

MANUAL REVIEW (MANDATORY)
Technical Writer or BA, with Architect

- Every field checked against the data dictionary
- Validation and error code IDs exist in the registry; none invented
- Each screen and field maps back to a requirement ID
- Business flow matches the approved register, not the AI version
- Reuse claims verified in the codebase before being accepted
- UI references use approved style classes only

EXIT GATE Finalised SFVD baselined in the document repository · requirement to screen to field matrix updated · released to Development and QA at the same moment.

ROLES R Technical Writer / BA A BA Lead C Architect, QA Lead, Product Owner I Development, Mobile, Release

Solution design and interface contracts

INPUTS GIVEN TO AI

- Finalised SFVD and the approved requirement register
- Current service map, domain boundaries and event topics
- Existing OpenAPI specifications for the affected services
- Database schema, naming standard and shared library index
- Non-functional targets, authorisation and audit model

WHAT AI PRODUCES

- Service impact map across all three surfaces
- OpenAPI contract draft with request, response and error shapes
- DTO, entity and sequence design for the main flows
- Database migration and rollback script drafts
- Offline and synchronisation notes for the mobile surface
- Build-versus-reuse recommendation with written justification

MANUAL REVIEW (MANDATORY) Architect with Tech Lead

- Service boundaries respected; no cross-service database access
- Contract backward compatible, or the version bumped deliberately
- Capability does not already exist in another service
- Base classes and shared libraries reused rather than re-created
- Indexes, migration and rollback reviewed with the DBA
- Authorisation, audit fields and PII handling all present

EXIT GATE Design note approved · API contract merged to the contract repository before any code is written · mobile and internal web build against that same contract · breaking-change log updated.

ROLES R Tech Lead A Architect C DBA, Security, QA Lead I All development teams

Development with AI inside our structure

REQ | SFVD | DESIGN | **DEV** | REVIEW | TEST CASE | TEST RUN | RELEASE | DEPLOY

INPUTS GIVEN TO AI

- Approved SFVD section, API contract and design note
- Coding standards, naming conventions and folder structure
- Shared library index and the UI style class catalogue
- Golden reference example for this surface
- The requirement ID this task delivers
- The same static analysis rule set the pipeline runs

WHAT AI PRODUCES

- Implementation placed inside the existing project structure
- Unit tests covering the new logic and its boundaries
- Configuration entries, i18n keys and migration scripts
- Inline documentation written in the house style
- Self-check note listing standards applied and reuse decisions

MANUAL REVIEW (MANDATORY)

The developer owns every line AI wrote

- Shared library searched before accepting any new utility or CSS
- Naming, layering and package placement match the standard
- No hardcoded values, no secrets, no leftover AI alternatives
- Exception handling, logging and audit follow the standard
- Query patterns checked for N+1 and missing pagination
- Requirement ID present in the branch name and every commit

EXIT GATE Pull request raised with the completed stage checklist · PMD, Sonar and coverage green on new code · self-review evidence attached · pull request kept under 400 lines or split.

ROLES R Developer A Tech Lead C Architect I QA, Release

Code review and static quality gates

INPUTS GIVEN TO AI

- Pull request diff and the completed developer checklist
- Coding standards and the shared library index
- Sonar baseline, duplication report and coverage delta
- API contract diff against the approved version
- Review checklist for this surface from the Context Pack

WHAT AI PRODUCES

- Plain-language summary of what the diff actually changes
- Standards deviation scan against our own rule set
- Duplicate logic matched against the shared libraries
- Missing or weak test detection on the changed paths
- AI assists the reviewer. It never approves and never merges.

MANUAL REVIEW (MANDATORY) Independent reviewer, never the AI operator

- Business logic read line by line against the SFVD
- Reuse opportunities enforced, not merely noted
- Contract adherence and backward compatibility confirmed
- Security, authorisation and data access reviewed
- Test adequacy judged on scenarios, not on coverage percentage
- Anything the reviewer cannot explain is sent back, not merged

EXIT GATE Human approval recorded · zero blocker and critical findings · duplication within threshold · new-code coverage at target · merged with the requirement ID in the message.

ROLES R Reviewer (peer or Tech Lead) A Tech Lead C Architect, Security I QA, Product Owner

Test case design from the approved SFVD

INPUTS GIVEN TO AI

- Finalised SFVD and the approved API contract
- Test case template and the test case ID convention
- Test data structure and the seed data specification
- Boundary and negative scenario pattern library
- Index of the existing regression suite
- Device, browser and user role matrix

WHAT AI PRODUCES

- Functional, validation, UI and business flow cases
- Integration and cross-surface cases derived from the contract
- Positive and negative scenarios with expected results
- Test data requirements mapped to the seed structure
- Requirement ID mapping and an automation candidate flag

MANUAL REVIEW (MANDATORY) QA Lead with the Business Analyst

- Every requirement ID and every field validation is covered
- No duplicate of a case already in the regression suite
- Expected results verified against the SFVD, never assumed
- Depreciation, label and audit scenarios checked by a domain expert
- Test data realistic, masked and compliant with policy
- Priority, regression and automation tags set deliberately

EXIT GATE Test suite approved in the test management tool · traceability matrix shows at least one case per requirement ID · suite baselined for this release.

ROLES R QA Engineer A QA Lead C Business Analyst, Developer I Product Owner, Release

Test execution and automation

INPUTS GIVEN TO AI

- Approved test suite and the seed data script
- Environment configuration and the test account matrix
- Automation framework patterns and the locator standard
- The existing automated regression pack
- Defect template and severity definitions

WHAT AI PRODUCES

- Automation script drafts following our page object pattern
- Locators, data-driven skeletons and reusable steps
- Defect descriptions drafted from failed runs
- Execution summary mapped back to requirement IDs

MANUAL REVIEW (MANDATORY) Automation Engineer with QA Lead

- Each script matches its approved case step for step
- Assertions genuinely assert the expected result
- No hardcoded data, no fixed waits, no environment coupling
- Tests independent and repeatable in any execution order
- Added to the regression pack with the correct tags
- Anything deferred recorded with a reason and a named owner

EXIT GATE Automation parity check passes: approved cases equal executed plus automated plus formally deferred · defect log complete and linked · evidence attached to the release record.

ROLES R Automation Engineer A QA Lead C Developer I Product Owner, Release

Build and release packaging

INPUTS GIVEN TO AI

- Merged code and the release scope drawn from Jira
- Version and branching policy
- Release note template and the previous release note
- Configuration and secret matrix per environment
- Migration scripts and the rollback plan

WHAT AI PRODUCES

- Release notes drafted from commits and requirement IDs
- Configuration difference summary between environments
- Deployment checklist and rollback step drafts
- Dependency and version change summary

MANUAL REVIEW (MANDATORY) Release Manager with DevOps

- Version and build number correct, unique and immutable
- Configuration difference verified against the environment matrix
- Migration dry run executed on a production-like copy
- Release notes reconciled against the traceability matrix
- Open defects accepted or deferred in writing, not by silence
- Approval recorded before the artifact is promoted

EXIT GATE Signed release package · traceability snapshot attached · artifact immutable and reproducible from the tagged commit.

ROLES R Release Manager A Delivery Manager C DevOps, Architect, QA Lead I All teams

Deployment, verification and closure

INPUTS GIVEN TO AI

- Approved and signed release package
- Deployment runbook and environment checklist
- Smoke pack generated from the traceability matrix
- Rollback plan and a confirmed backup
- Monitoring dashboard and alert thresholds

WHAT AI PRODUCES

- Smoke checklist assembled from the traceability matrix
- Post-deployment log and metric anomaly summary
- Draft closure note and known-issues list

MANUAL REVIEW (MANDATORY)
DevOps with QA and Product Owner

- Pre-deployment validation and backup confirmed by a named person
- Steps executed from the runbook, never improvised under pressure
- Smoke pack executed and signed by QA, not by the deployer
- Monitoring watched for the agreed window after release
- Business sign-off recorded against the release
- Rollback decision owner named before deployment starts

EXIT GATE Smoke pack passes in full · monitoring clean for the agreed window · release closed and the traceability matrix frozen as the audit baseline for this version.

ROLES R DevOps A Delivery Manager C QA Lead, Product Owner, Architect I All teams

Defect handling: where AI stops and the engineer starts

P1 and P2 production defects

No AI at any point. Root cause analysis, fix and regression test are written by the engineer, reviewed by a second engineer.

P3, P4 and pre-release defects

Root cause analysis is manual and approved first. AI may then draft the agreed fix, within the normal stage 4 and 5 gates.

Never, at any severity

Asking AI to make a symptom disappear before a human has written down what actually caused it.

MANDATORY FOR EVERY DEFECT, WHATEVER THE SEVERITY

- Root cause recorded: what broke, why it was not caught, and which gate missed it.
- A failing test is written before the fix is made, and kept in the regression pack.
- Defect linked to its requirement ID and to the release that introduced it.
- Fix reviewed by someone other than its author, under the stage 5 gate.
- The gate that let the defect through is strengthened, or a waiver is logged with an expiry.
- If the root cause was a context gap, the Context Pack is updated in the same week.

Decision needed today: our current rule bans AI for all bug fixing. This refinement keeps the ban where the risk is highest and allows AI on low-severity work once a human has written the root cause. The team needs to confirm the boundary.

06

Verification and governance

Traceability, standardised verification, automated gates and who owns what

One unbroken chain: nothing exists in this product without an ID



HOW THE CHAIN STAYS ALIVE

- The requirement ID is a mandatory field in Jira, the SFVD header, the test case, the branch name and the commit message.
- The matrix is generated nightly from those systems — it is never a spreadsheet someone remembers to update.
- A link-check job fails the release build if any requirement has no test case, or any commit has no requirement.
- AI stamps the IDs, the reviewer confirms them, the job proves them.

WHAT THE CHAIN BUYS US

- Coverage can be proven at audit in minutes instead of days.
- The impact of a change is visible across all three surfaces before it is made.
- The next enhancement starts from evidence rather than archaeology — which is how we keep future work routine instead of critical.
- A defect can be traced back to the gate that missed it, so the gate gets fixed, not the person.

This single control answers three of our six gaps: traceability loss, silent omission and review debt.

Verification is standardised: four levels close every gate

L1 — **Machine checks**
Lint, PMD, Checkstyle, Sonar, contract diff, coverage, duplication, secret scan and the ID link check. Objective, instant and blocking.

L2 — **Role checklist**
The stage checklist from the Context Pack. Every item ticked by the named role, with evidence attached — not a verbal confirmation.

L3 — **Independent review**
A second human, never the person who ran the prompt, reads the output against the approved input from the previous stage.

L4 — **Recorded sign-off**
A tool transition carrying the artifact version, the Context Pack version and the reviewer name. This is the audit trail.

The evidence rule: a gate opens only when all four levels are complete and recorded. Three out of four is not a gate — it is a hope.

The automated gates — what blocks what, and at which point

Gate	Tooling	Threshold	Blocks
Standards and naming	PMD, Checkstyle, custom naming rules	Zero violations in new code	Pull request merge
Code quality	SonarQube quality gate	Zero blocker or critical findings	Pull request merge
Test coverage	SonarQube new-code coverage	At or above the agreed target on new code	Pull request merge
Duplication	SonarQube / CPD	Within the agreed limit on new code	Pull request merge
Interface contract	OpenAPI diff against the approved spec	No breaking change without a version bump	Pull request merge
Security	Dependency and secret scanning	No high findings, no secrets committed	Pull request merge
Traceability	Nightly link-check job	Every requirement linked to code and tests	Release build
Test parity	Test management export comparison	Approved equals executed, automated or deferred	Release build
Smoke verification	Smoke pack generated from the matrix	Full pass inside the monitoring window	Release closure

Exact thresholds are set by the Council and reviewed quarterly. They start in warning mode during the pilot and become blocking at day 60, so the team tunes them rather than fights them.

Who does what: one accountable name per stage

Stage	Runs the AI step	Accountable	Reviews and consulted	Informed
1 Requirements	Business Analyst	**Product Owner**	Architect, QA Lead	Dev, Release
2 SFVD	Technical Writer / BA	**BA Lead**	Architect, QA Lead, PO	Dev, Mobile, Release
3 Design and contract	Tech Lead	**Architect**	DBA, Security, QA Lead	All dev teams
4 Development	Developer	**Tech Lead**	Architect	QA, Release
5 Code review	Reviewer (peer)	**Tech Lead**	Architect, Security	QA, Product Owner
6 Test design	QA Engineer	**QA Lead**	Business Analyst, Developer	PO, Release
7 Test execution	Automation Engineer	**QA Lead**	Developer	PO, Release
8 Build and release	Release Manager	**Delivery Manager**	DevOps, Architect, QA Lead	All teams
9 Deploy and close	DevOps	**Delivery Manager**	QA Lead, PO, Architect	All teams

The one rule that cannot bend: the person who ran the AI prompt is never the person who signs the gate for that output. Every stage above keeps those two names apart.

Governance: keeping all three surfaces in step as we scale

AI GOVERNANCE COUNCIL

- Architect as chair, with QA Lead, Delivery Manager and a security representative.
- Thirty minutes, weekly. Owns the Context Pack, the gate thresholds and every waiver.
- Reviews the metrics pack monthly and decides what changes.

CONTEXT PACK CHANGE CONTROL

- Every change is a pull request with two approvals and a version bump.
- A short release note goes to all three surface teams on the same day.
- The new version is stamped on every artifact produced after it lands.

MONTHLY DRIFT AUDIT

- A ten percent sample of AI-assisted merges is scored against the stage checklists.
- Findings become Context Pack improvements, not individual blame.
- Repeat findings escalate to a blocking gate at the next review.

WAIVERS AND ESCALATION

- Any deviation needs a written waiver with a named owner and an expiry date.
- Schedule pressure never bypasses a gate — only the Council can waive one, in writing.
- Open waivers are reviewed at every release and cleared or renewed.

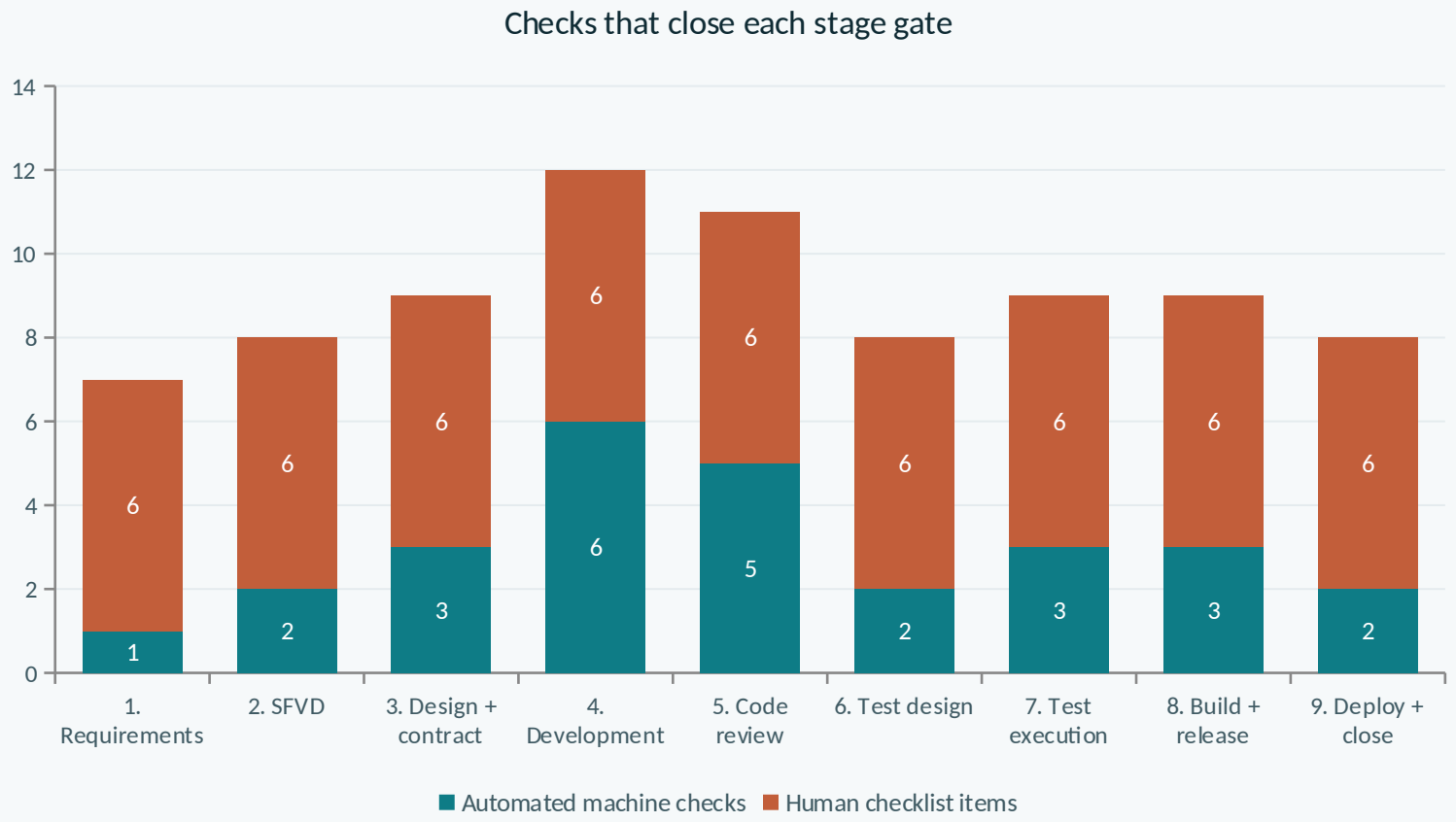
Weekly Council, 30 min

Monthly Audit and metrics

Quarterly Refresh prompts

Each release Matrix frozen

How we will know it is working



Human review is constant across all nine stages. Automation varies — it carries the heaviest load exactly where AI writes the most.

REVIEWED MONTHLY BY THE COUNCIL

- Traceability matrix completeness — target 100%
- Automation parity — target 100%
- Commits carrying a requirement ID — target 100%
- Gates closed with recorded evidence — target 100%
- New-code coverage and duplication against the agreed limits
- Review findings per AI-assisted pull request — expected to fall as the Context Pack matures
- Defects escaping to production per release
- Cycle time per stage — this is where the AI benefit shows up

Baseline is measured in sprint one. Targets are commitments, not estimates.

07

Rollout and decisions

Ninety days, three pilots, four decisions we need from you today

ROLLOUT

Ninety days from today, in three steps

Days 0 to 30

Foundation

- Publish Context Pack v1.0: standards, naming, folder structure, shared library index, style classes, test data specification.
- Write the nine stage prompt templates and the nine review checklists.
- Name the Council and agree the initial gate thresholds.
- Pick one pilot feature on each of the three surfaces.

Days 31 to 60

Pilot and prove

- Run the full nine-stage flow end to end on the three pilot features.
- Turn the CI gates on in warning mode and measure the baseline.
- Tune the Context Pack from every review finding, weekly.
- Hands-on working session for the whole team, after this one.

Days 61 to 90

Enforce and scale

- Gates move from warning to blocking.
- Traceability and parity checks become mandatory for the release build.
- First monthly drift audit runs; golden examples published.
- All new work follows the framework; retrospective produces Context Pack v1.1.

Nothing stops while this is built. Current work continues; the framework is applied to new features first, and to existing modules as they are touched.

THE SHORT VERSION

What we are asking every one of you to do differently

DO

- Start from the approved artifact of the previous stage, never from memory.
- Load the Context Pack and use the stage prompt template — do not improvise a prompt.
- Make AI declare its assumptions, then review that list first.
- Search the shared library before accepting any new utility, CSS class or base method.
- Review in small batches: under 400 lines, with the SFVD open beside the diff.
- Stamp the requirement ID on the branch, the commit, the test case and the defect.
- Raise a context gap as a pull request to /ai-context — fix it once, for everyone.

DO NOT

- Do not paste a requirement straight into a free-form prompt.
- Do not accept code or a test case you could not explain to a reviewer.
- Do not let AI invent a field, a validation, an error code or an API shape.
- Do not create a new utility, style class or service before searching for one.
- Do not approve your own AI output — the gate needs a second name.
- Do not use AI on a production P1 or P2 fix before a human has written the root cause.
- Do not skip a gate because the date is close. Ask the Council for a waiver instead.

Print this slide. It is the whole framework in fourteen lines.

CLOSING

What we need from you

1

Adopt the Context Pack from Monday

Every prompt for a deliverable starts from /ai-context and a stage template.

2

No stage output moves without a signed gate

Four levels of verification, a named reviewer who is not the prompt operator.

3

Treat a context gap as a defect

Raise a pull request against the pack so the fix reaches everyone at once.

DECISIONS NEEDED TODAY

- The AI boundary for bug fixing: confirm the P1 and P2 rule.
- Initial gate thresholds for coverage and duplication.
- The four names on the AI Governance Council.
- The three pilot features, one per surface.

We are not slowing AI down. We are making its output something we can sign our names to — so that the enhancement we are asked for in year three is routine work, not a critical project.

Next: Context Pack v1.0 review session — one week from today.